

BTBench: A Benchmark for Comprehensive Binary Translation Performance Evaluation

Xinyu Li*, Yanzhi Lan*, Gen Niu*, Feng Xue[†], Fuxin Zhang*

State Key Laboratory of Computer Architecture, Institute of Computing Technology, Chinese Academy of Sciences

University of Chinese Academy of Sciences, China

*{lixinyu20s, lanyanzhi22b, niugen18z, fxzhang}@ict.ac.cn

[†]xuefeng16@mails.ucas.ac.cn

Abstract—Binary translation serves as a fundamental technology for instruction set emulation, system virtualization, runtime instrumentation, and numerous other applications. Many techniques have been proposed to enhance the efficiency of binary translation systems. However, imprecise performance evaluation leads to potential performance shortcomings of binary translators in real-world applications. Previous studies primarily employ CPU benchmarks, which may overlook performance issues specific to binary translators and fail to guide for optimizing binary translation.

To address this issue, we propose a new benchmark suite named BTBench(Binary Translation Benchmark), which provides a convenient, portable, and comprehensive solution. BTBench takes into account the inherent attributes of binary translators, such as translation and code-cache lookup overhead. We carefully select benchmarks that offer comprehensive coverage and align with real-world application scenarios. To validate the effectiveness of BTBench, we conducted rigorous experimentation on four widely-used binary translators. The analysis of the results reveals that, compared to existing CPU benchmarks, BTBench is better suited for identifying potential performance shortcomings, providing invaluable insights for future optimization efforts. The BTBench benchmark suite is publicly available¹.

Index Terms—Binary Translation, Performance Evaluation, Benchmark

I. INTRODUCTION

Binary translation (BT) has a wide range of application scenarios, such as software simulation [9], [16], [21], [54], program runtime analysis [11], [17], [49], [52], debugging [18], [26], [50], and dynamic optimization [7], [11]. With the advent of the golden age of computer architecture [35], new instruction set architectures (ISAs) have been proposed, such as RISC-V [73], AArch64 [76], and LoongArch [48], bringing continuous vitality to the field of computer architecture. New ISAs require binary translation for testing, development, and ecosystem support.

Binary translation often suffers from significant performance overhead, thereby limiting its applicability. Consequently, researchers have explored various optimization methods to mitigate this issue [28], [37], [54], [63], [71]. Benchmark evaluations play a vital role in obtaining performance metrics for binary translation, providing essential feedback to identify and address significant issues and guide future design improvements. However, there is a notable absence

of benchmarks specifically tailored for binary translation. We analyzed a total of 289 literature on binary translation optimization. Among them, 75.1% utilized SPEC CPU [67] as the benchmark for performance testing, while a smaller portion used simpler CPU benchmarks, and some employed vectorization or multi-thread related benchmarks. Therefore, SPEC CPU, representing CPU benchmarks, plays a dominant role in performance testing for binary translation.

An effective benchmark suite for binary translation necessitates careful consideration of both the binary translator’s design and the traits exhibited by the actual applications. CPU performance evaluation benchmarks, such as SPEC CPU, might not comprehensively capture potential performance bottlenecks that could be present in binary translators. It is important to acknowledge that SPEC CPU has limited application coverage, lower translation overhead, and reduced code complexity in the context of binary translation. The variations in performance results in SPEC CPU benchmarks arise from the discrepancies between binary translation and physical CPU execution. For instance, applications with self-modification behavior in JavaScript engines may incur minor performance loss on the physical CPU, but they can result in substantial performance degradation for the binary translator [34], [72]. Inadequate performance evaluation hinders the fair comparison of binary translators and may lead to inconsistencies in real-world performance. As such, a tailored benchmark is indispensable for comprehensively assessing the performance of binary translators and identifying crucial areas for optimization and refinement.

In this paper, we present BTBench, an innovative and highly flexible benchmark suite specifically designed for binary translators. It comprehensively covers various applications from the perspective of programming languages while taking into account the components of binary translation. BTBench includes 21 carefully selected workloads that cover a wide range of areas, such as large instruction footprint programs, code with complex branch behavior, and self-modified applications, closely resembling real-world complex applications. Unlike conventional benchmarks, BTBench effectively exposes performance limitations unique to binary translation, thus offering invaluable insights to further enhance binary translators. With its comprehensive and representative capabilities, BTBench is expected to play a vital role in guiding the future development

¹The BTBench is available at <https://github.com/rwrhx/btbench>.

and optimization of binary translation technologies.

The main contributions of this paper are as follows:

- We have identified the limitations in the existing benchmarks used in binary translation. In response, we have developed a set of criteria that are more appropriate for selecting benchmarks for binary translation.
- We deliver an open-source benchmark suite named BT-Bench, a novel and portable benchmark specifically tailored for binary translation scenarios. BTBench provides a more objective and accurate assessment of binary translation system performance. To the best of our knowledge, this is the first benchmark explicitly developed for binary translation.
- We conducted an extensive analysis using BTBench to examine the prevailing binary translators. The analysis uncovered notable performance differences in these translators, providing valuable insights for future optimization.

The organization of the rest of this paper is as follows. Section II provides an introduction to BT and its performance evaluation. Section III details the design of the BTBench benchmark. Section IV conducts a comprehensive analysis of BTBench and SPEC CPU2017 [12] benchmarks on four binary translators. Section V and Section VI introduce the related work and presents the summary of the paper, respectively.

II. BACKGROUND AND MOTIVATION

This chapter offers an overview of the fundamental components of binary translation and presents an overview of the current benchmarks used for performance evaluation. Furthermore, it clarifies the underlying motivation behind this study.

A. Binary Translation

A binary translator takes executable binary code as input and translates it into functionally equivalent host binary code, composed of similar components [9] [49] [8] [32], as shown in Figure 1. The translated ISA is referred to as the guest ISA and the underlying platform on which the translation occurs is known as the host ISA. The ability of cross-ISA execution enables the use of binary translation in the ISA emulation and cross-platform software emulation. Moreover, the translator can observe and control the program's execution. This allows for detailed analysis of the program's execution and the incorporation of necessary modifications. As a result, BT is applied in program instrumentation and security. The key components of a binary translator are as follows.

1) *Translator*: The translator assumes the responsibility of decoding, translating, and optimizing guest instructions. While some translators utilize extensive profiling information and time-consuming compilation optimization techniques to generate highly efficient code [24], [37], others opt for simpler conversion methods to minimize translation time [9].

2) *Executor*: The executor is responsible for the context switching of the binary translator and for executing the code generated by the translator. The performance of the generated code primarily determines the performance of the executor.

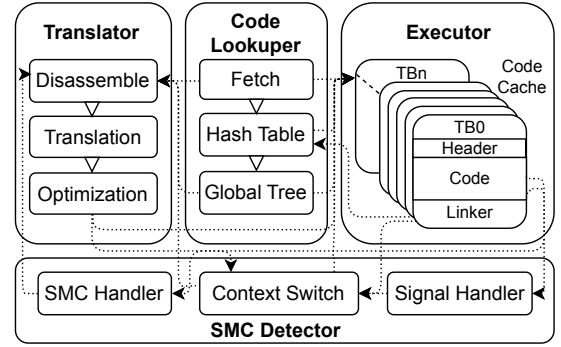


Fig. 1: The Components of Binary Translator

3) *Code Lookuper*: Code lookuper is responsible for locating the host code address based on the guest code address. This mapping information of guest code address and host code address is typically cached using a hash table. To enhance the code lookup performance, several hardware and software solutions have been proposed [20], [43].

4) *SMC Detector*: The SMC detector is responsible for maintaining consistency between the host code and the guest code. Certain Just-in-Time (JIT) programs, such as JavaScript and PyPy applications, exhibit self-modifying code (SMC). The presence of SMC disrupts the inherent mapping between guest code and host code, thus necessitating BT to ensure its guarantee. Various techniques have been proposed to optimize the detection and expedite the re-translation of SMC [72].

B. Binary Translation Performance Evaluation

The performance of a binary translator is typically represented by its efficiency. Efficiency is obtained by dividing the execution time of a program run on the BT environment by the execution time of the same program run directly on the host, as shown in Formula 1.

$$Efficiency_{BT} = \frac{RunTime_{Native}}{RunTime_{BT}} \quad (1)$$

For example, if a guest program running on the host through binary translation takes 10 seconds while compiling directly for the host and running natively requires 6 seconds, then the efficiency is 60%.

The performance of binary translation is determined by the components working together. However, achieving both fast translation and high-performance translated code can be challenging. The performance of the translator involves a trade-off between the speed and performance of the translation.

C. Analysis of Benchmark for Binary Translation

The benchmark for binary translation should cover the usage scenarios of binary translation. Binary translators used for ecological migration are widely used on the server side, such as ExaGear [39], and on the client side, such as Rosetta 2 [4]. Additionally, binary translators used for program hardening and instrumentation run various types of programs, such as Intel Pin. This implies that any real-world application can

serve as input to binary translators. Hence, it is crucial for the design of BTBench to ensure that the workloads cover a diverse range of applications.

Furthermore, the benchmark used for binary translation should thoroughly take into account the components of binary translation and the factors that impact performance.

It is possible to use any benchmark suite to assess the performance of binary translators. However, among the various studies on binary translation, SPEC CPU has emerged as the dominant choice [3], [19], [24], [33], [37], [54], [59]. SPEC CPU is considered a significant benchmark standard for evaluating CPU performance. However, based on our observations and experiments, we have found that SPEC CPU exhibits inherent limitations in the realm of binary translation. These limitations encompass the following:

1) *Low Real Applications Coverage*: SPEC CPU solely focuses on CPU usage scenarios and overlooks the specific characteristics of binary translation, resulting in the absence of real-world application programs that significantly influence binary translation. Firstly, it excludes self-modifying programs. In hardware CPUs, the consistency of data cache and code cache is upheld through hardware mechanisms, hence restricting the overhead caused by pipeline pauses for self-modifying code (SMC). However, in the realm of binary translation, the translator assumes the entire responsibility of maintaining consistency, leading to substantial overhead [34], [72]. SPEC CPU lacks any SMC behavior, rendering it incapable of capturing and assessing the performance impact of SMC on binary translators. Secondly, SPEC CPU only encompasses programs comprised of C/C++/Fortran programming languages, whereas real-world programs incorporate diverse programming languages. GitHub statistics reveal a higher emphasis on JavaScript, Go, and Rust code [25], which suggests their prevalence in real-world applications.

2) *Low Translation Overhead*: The translation overhead is defined by Formula 2. When the translation algorithm of a binary translator is fixed, the time it takes for translation primarily depends on the size of the input instruction footprint. The SPEC CPU benchmark has a smaller instruction footprint and a longer runtime, resulting in a low translation overhead.

$$Overhead_{Translation} = \frac{RunTime_{Translation}}{RunTime_{Native}} \quad (2)$$

Previous research has indicated that the translation overhead on SPEC CPU is minimal [10]. Our measurements reveal that during the execution of SPEC CPU 2017 ref input using Pin, the translation only consumes approximately 2.2% of the total time. When running specific applications, such as Octane [30] with Intel Pin, there is a notable increase in overhead, reaching up to 104% as per our test results. Consequently, the performance of translation is crucial for binary translation, yet it seems to be disregarded by the SPEC CPU benchmark.

3) *Less Code Complexity and Smaller Code Footprint*: SPEC CPU benchmarks often simplify the source code [65], [66] or combine workload input [64] for ease of use and

testing. As a result, the SPEC CPU hotspot code exhibits a comparatively simplified structure, enabling BT to readily gather profiling information and exert less strain on the optimization path of BT. Furthermore, owing to its limited code footprint, the program displays relatively straightforward branch behavior, resulting in diminished demands on the code lookup algorithm.

III. BENCHMARK SUITE DESIGN

This section elucidates the methodology utilized to gather BTBench benchmarks, provides detailed information about BTBench workloads, and outlines the execution harness.

A. The Benchmarks Selection

In light of the limitations of existing benchmarks, the following two criteria should be considered for binary translation benchmarks. **(a)** Encompassing a diverse array of applications close to the real world. **(b)** Taking into account the characteristics of binary translators. We implemented these criteria by utilizing the following strategies.

1) *Wide Range of Programming Languages and Commonly Utilized Domains*: Programs originate from various programming languages, each with their distinct areas of application. For example, C/C++ is used for developing operating systems and various system tools, while JavaScript is used for interactive web development. Therefore, by incorporating a wide range of programming languages and their primary usage domains, the benchmark can cover the majority of scenarios found in diverse programs. Hence, BTBench includes seven programming languages from different domains. According to the TIOBE Index (a programming language popularity ranking) [69], all seven selected programming languages are among the top ten widely used languages. According to the statistics from GitHub repositories, the top 100 repositories in JavaScript, Go, Rust, C++, and C have the highest number of stars [25], which indicates that these programming languages are widely used in real-world applications. Covering mainstream programming languages and their commonly utilized domains indicates the coverage of a significant portion of application programs.

2) *Programming Languages with Different Compile Mode*: Considering that the main purpose of a binary translator is to translate input guest binary instructions into semantically equivalent host binary instructions, the source of input binary instructions can be categorized into three types: **(a)** Compiled languages such as C and Go, where the instructions are generated by the corresponding compilers during the compile phase. **(b)** Interpreted languages like Python, where the instructions input to the binary translator are the instructions of the interpreter itself. **(c)** JIT-style languages like JavaScript, which include both the instructions of the JIT engine itself and the instructions dynamically generated by the JIT engine, such as the V8 JavaScript engine.

From this perspective, using different modes of languages is sufficient to cover a wide range of inputs for the binary translator. Therefore, in its design, the BTBench benchmark not only covers compiled languages like C and C++ but also includes

interpreted languages such as Python and PHP, as well as just-in-time compiled languages like JavaScript and PyPy. By incorporating programs written in diverse programming languages with distinct compilation modes, it can effectively cover the inputs to binary translators and even explore the impact of different language instruction generation patterns on the binary translator.

3) *Unmodified Real World Applications*: Typical benchmarks often simplify the source code of a program or select a single algorithm for testing to highlight the program’s core operations and enable easier debugging and usage. However, as mentioned in section II-C, binary translators are sensitive to the code footprint and complexity of the input program. In real-world applications, there is typically a significant amount of initialization code and various complex functionalities, resulting in large code footprints and high complexity. To accurately simulate real-world scenarios, BTBench aims to obtain performance results that closely resemble actual applications by minimizing modifications to the source code.

BTBench takes into account the wide application of real-world scenarios and the performance impact of binary translators, such as different programming languages and source code lengths. This strategic approach enables BTBench to conduct a comprehensive and realistic performance assessment of binary translators, thereby enhancing the comprehensiveness and authenticity of its evaluations.

B. The Benchmarks Constituent

The workload initially collected comprises two components. The first component is gathered from popular language-specific domains on the open-source platform GitHub, along with their respective usage cases. For instance, *mdbook* [61] is utilized for the Rust programming language. The second component is derived from well-established benchmarks in different languages, such as *Octane* [30] for JavaScript. ALL workloads underwent performance analysis and similarity analyses, employing principal component analysis (PCA) as explained in Section IV-H. Subsequently, a total of 21 benchmarks with distinct performance characteristics were selected. These selected benchmarks cover 7 different domains of programming languages and 3 different compilation modes. Moreover, they span a wide range of domains and application scenarios, thus potentially serving as representatives of real-world applications and providing a comprehensive evaluation of binary translator performance. Table I displays all the BTBench workloads.

It is worth noting that BTBench and SPEC CPU have some common workloads, such as gcc, but the workloads and inputs in BTBench are different from SPEC CPU. The gcc in SPEC CPU modifies the source code and uses larger inputs to highlight the computational part of the program [64], while the gcc in BTBench uses unmodified source code and shorter inputs to match real-world scenarios, so they have some performance differences.

To showcase the potential variations in performance resulting from binary translation across various components, we

employed the powerful instrumentation tool, Pin, to collect statistical data on the pertinent parameters of BTBench. These parameters encompass:

Number of Instructions Footprint determines the number of instructions that the binary translator must translate. It directly impacts the translation time of the binary translator. To obtain this parameter, we counted the distinct instructions executed by the program utilizing the Pin tool.

Number of Indirect Branch Targets reflects the varying destinations for indirect jump instructions within the program. This parameter has a direct impact on the complexity and time required for code lookup by the binary translator. We utilized the Pin tool to examine the number of distinct indirect jump targets.

Count of SMC signifies the number of self-modifying code occurrences within the program. It significantly impacts the BT’s ability to efficiently re-translate and handle SMC, affecting overall performance. To acquire this metric, we directly utilized Pin’s APIs, specifically designed for detecting and monitoring SMC behavior.

C. The Benchmark Harness

Any binary translator capable of executing Linux user programs can run BTBench for performance evaluation, regardless of the ISA. Furthermore, BTBench can easily be extended to other operating systems in the future. Additionally, BTBench is suitable for performance testing of full system binary translators; however, given the limited availability of full system binary translators, this study primarily focuses on user mode binary translators.

We have devised a comprehensive framework that encompasses benchmark configuration of the binary translator, execution, and score computation. Moreover, the framework provides support for multiple hooks to facilitate the collection of performance information. Notably, performance analysis tools such as Linux Perf and instrumentation tools like Pin can be utilized to gather various performance statistics, allowing for further analysis.

IV. EVALUATION

A. Evaluation Setup

To ensure the optimal versatility and performance of BTBench, we conducted a thorough evaluation encompassing analysis and comparisons. Specifically, we selected two major ISA platforms, namely AMD64 and AArch64. The performance evaluation was carried out using four widely employed binary translators, which encompass various use cases such as instruction set emulation, instrumentation, and ecological migration. These binary translators are introduced below.

QEMU [9] is widely used in both industry and academia areas [37], [71], [78]. QEMU utilizes an intermediate language to provide support for various host ISAs and guest ISAs. However, the performance of QEMU is low due to its limited optimization techniques. For the sake of experimental convenience, we have chosen to adopt a scenario in which both the host and guest are AMD64. Under the same instruction

TABLE I: Benchmark Description

Benchmark	Programing Language	Compile Mode	Application Domain	# Instruction Footprint	# Indirect Branch Target	# SMC
gcc [68]	C/C++	compiled	compiler	1.2M	5.9K	-
x264	C	compiled	video encoder	28K	451	-
xz [15]	C	compiled	data compression	154K	1.4K	-
nlohmann_json [47]	C++	compiled	json encode/decode	139K	2.1K	-
xalan	C++	compiled	XML transformer	29K	255	-
buntdb [6]	Go	compiled	database	40K	173	-
gse [23]	Go	compiled	NLP and text segmentation	57K	251	-
jit_zend [57]	PHP	JIT	web server	223K	2.4K	18K
adatron	PyPy	JIT	machine learning	367K	2.0K	161
aes	PyPy	JIT	data encryption	371K	1.9K	502
navierstokes [30]	JavaScript	JIT	fluid mechanics	618K	11K	4.4K
pdfjs [30]	JavaScript	JIT	pdf reader	808K	13K	162K
script_compiler [30]	JavaScript	JIT	programming language	1.0M	13K	768K
zlib [30]	JavaScript	JIT	data compression	606K	9.8K	342
phpbench [58]	PHP	interpreted	web server	171K	1.9K	-
zend [57]	PHP	interpreted	web server	136K	1.6K	-
adatron	Python	interpreted	machine learning	170K	1.2K	-
aes	Python	interpreted	data encryption	166K	1.2K	-
mdbook [61]	Rust	compiled	document transformer	192K	1.3K	-
rust-bio [44]	Rust	compiled	bioinformatics	19K	189	-
svgbob [14]	Rust	compiled	picture processing	83K	489	-

set, the performance of QEMU is not affected due to the translation through intermediate language and lack of specific optimization.

Pin [49] is a binary instrumentation tool developed by Intel Corporation. It is utilized for the analysis and modification of binary application execution. Besides, Pin can generate traces for various simulators, including ChampSim [29] and Sniper [13]. Pin has been the basis for various tools in computer architecture research, like PinPlay [56] and ELFies [55].

Rosetta 2 [4] plays a crucial role in the transition from Intel-based Macs to the new Apple Silicon architecture, serving as a pivotal binary translation tool developed by Apple. To facilitate testing, we employed the Linux version of Rosetta 2 [5]. To simulate x86 on ARM architecture, the correct implementation of x86-TSO memory ordering is essential. Apple Silicon chips have hardware support for this purpose. However, due to the absence of performance profiling tools on the Apple Silicon platform, we opted for the more generic ARM processor(Yitian 710) which lacks hardware support for x86-TSO. Subsequently, Rosetta 2 does not offer compatibility with x86-TSO. However, no instances of correctness issues have arisen.

ExaGear [38], developed by Huawei, is a user-mode binary translator. Its purpose is to enable the execution of applications developed for x86 processors on ARM processors. ExaGear is primarily used for server-side ecological migration. ExaGear incorporates x86-TSO by utilizing memory barriers. Nonetheless, this functionality remains disabled by default due to the significant performance decline it induces. Consequently, this option was not utilized during the experiments.

The configuration of the experimental platform is presented in Table II and III. QEMU and Pin were executed on the AMD64 platform, while Rosetta 2 and ExaGear were executed on the AArch64 platform.

Given the prevalent use of SPEC CPU in binary translation performance evaluations and the lack of dedicated benchmarks for binary translation previously, SPEC CPU was selected as the only benchmark for comparison. We conducted a comprehensive comparison of the results obtained by running BTBench and SPEC CPU2017 *ref* workload on each binary translator. To minimize the impact of architectural differences, the benchmarks were conducted using identical source code, compiler, and compilation options. BTBench and SPEC CPU benchmarks were compiled for the AMD64 and AArch64 architectures and were executed both natively and using each translator. Both benchmarks were run three times to ensure consistent and reliable results.

TABLE II: Evaluation Host Platform

	AMD64	AArch64
CPU	Intel 12700k	Yitian 710
Architecture	Goldencove	Neoverse n2
OS	Ubuntu 22.04.2	
C Compiler	gcc 11.4	
Rust Compiler	rustc 1.66.1	
Go Compiler	go 1.18.1	
Python Interpreter	Python 3.11.5	
Python JIT Engine	PyPy 7.3.9	
PHP Interpreter	PHP 8.1.2	
PHP JIT Engine	PHP 8.1.2	
JavaScript Engine	V8 7.8.279.23	

TABLE III: Evaluated Binary Translators

	QEMU	Pin	Rosetta	ExaGear
Vendor	Open Source	Intel	Apple	Huawei
Host ISA	AMD64	AMD64	AArch64	AArch64
Guest ISA	AMD64	AMD64	AMD64	AMD64
Version	6.2.0	3.27	310	2.0.0.1

B. Overall Performance

The overall performance is presented in Table IV and Table V. The duration of both binary translator execution and native program execution was recorded to calculate the performance using Formula 1. Additionally, Figure 2 illustrates the overall performance comparison of the results. Based on the results obtained from executing BTBench, ExaGear achieved the highest performance, followed by Rosetta, Pin, and QEMU.

TABLE IV: Overall Performance on SPEC CPU2017

Benchmark	QEMU	Pin	Rosetta 2	ExaGear
500.perlbench	13.7%	58.6%	56.3%	72.7%
502.gcc	19.9%	20.4%	69.6%	77.1%
505.mcf	38.0%	86.2%	84.4%	87.9%
520.omnetpp	25.8%	78.7%	74.2%	73.3%
523.xalancbmk	30.8%	83.0%	78.3%	86.2%
525.x264	7.0%	82.2%	54.1%	58.0%
531.deepsjeng	18.8%	65.7%	66.0%	76.9%
541.leela	24.0%	69.1%	69.8%	73.5%
548.exchange2	18.5%	77.2%	131.6%	118.7%
557.xz	40.5%	85.3%	84.2%	93.1%
503.bwaves	4.5%	94.2%	67.4%	62.6%
507.cactuBSSN	1.8%	76.9%	79.3%	86.4%
508.namd	3.7%	94.2%	51.0%	58.6%
510.parest	6.1%	54.4%	56.5%	47.3%
511.povray	4.1%	58.1%	55.2%	60.4%
519.lbm	3.6%	98.8%	58.3%	65.8%
521.wrf	5.0%	89.2%	71.3%	74.5%
526.blender	9.3%	90.3%	64.5%	69.4%
527.cam4	4.8%	58.0%	63.5%	68.3%
538.imagick	5.3%	89.5%	53.0%	62.7%
544.nab	7.0%	91.7%	62.7%	71.0%
549.fotonik3d	6.2%	107.2%	86.3%	87.5%
554.roms	4.8%	94.6%	77.7%	81.1%
min	7.0% ^a	20.4%	51.0%	47.3%
GEOMEAN	21.3%	75.1%	68.6%	73.1%

^a QEMU utilizes soft-float for precise floating point emulation, resulting in low performance. In the data, only the integer workloads are used.

However, notable differences were observed in the performance of the binary translator on SPEC CPU2017. Our evaluation uncovered several significant findings regarding the performance of binary translators on BTBench compared to SPEC CPU2017.

a) *Performance Decline*: We observed a substantial decrease in the performance of all binary translators when tested on BTBench, as shown in Figure 2. The largest decline was observed in Pin, with a decrease of 36.4% (from 75.1% to 38.7%). BTBench, due to its design, offers a more realistic evaluation of performance for binary translators. Therefore, it is reasonable to conclude that relying solely on SPEC CPU benchmarks may result in an overly optimistic assessment of binary translator performance.

b) *Deteriorated Worst-case Performance*: The worst-case performance for four binary translators significantly worsened. For example, ExaGear’s worst-case performance decreased from 47.3% to 22.4%. This suggests that BTBench could have exposed performance bottlenecks in the binary translator. The specific reasons will be examined in the follow-

TABLE V: Overall Performance on BTBench

Benchmark	QEMU	Pin	Rosetta 2	ExaGear
gcc	14.5%	18.2%	64.5%	74.4%
nlohmann_json	17.0%	48.3%	67.0%	74.9%
xalan	8.8%	35.1%	48.7%	58.7%
x264	1.7%	64.9%	42.6%	49.1%
xz	45.3%	87.8%	85.2%	105.1%
buntdb	34.9%	79.5%	82.3%	86.9%
gse	21.5%	69.5%	67.5%	74.2%
jit_zend	15.2%	49.6%	63.7%	22.4%
adatron	10.5%	13.4%	79.2%	58.2%
aes	23.9%	20.7%	77.6%	66.2%
navier_stokes	8.6%	52.5%	54.5%	70.0%
pdfjs	17.5%	21.7%	53.4%	58.4%
script_compiler	14.1%	10.3%	45.0%	49.2%
zlib	23.0%	18.8%	50.8%	64.6%
phpbench	12.5%	42.3%	49.8%	61.8%
zend	12.0%	38.7%	46.0%	67.6%
adatron	9.3%	42.6%	37.9%	54.8%
aes	14.1%	45.1%	43.1%	57.0%
mdbook	19.6%	63.5%	60.4%	73.6%
rust-bio	16.6%	45.3%	59.7%	58.1%
svgbob	20.6%	82.0%	75.0%	61.1%
min	1.7%	10.2%	37.9%	22.4%
GEOMEAN	14.7%	38.7%	58.1%	61.9%

ing sections. The lowest performance can reflect the bottleneck of binary translators, and BTBench performs better.

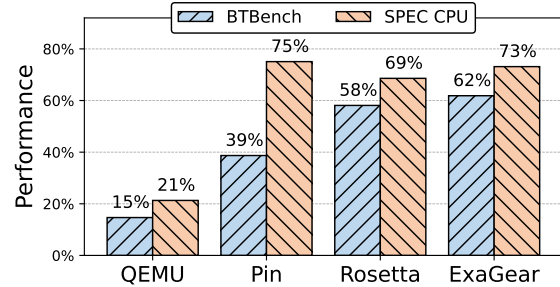


Fig. 2: Geomean of Overall Performance

We have illustrated the performance distribution to enhance the contrast between the result of the binary translator on BTBench and SPEC CPU2017. Figure 3 depicts this distribution, utilizing data from Table V and Table IV. Firstly, the results indicate that the performance center of BTBench is skewed towards the left, highlighting significant performance degradation when running this benchmark. Secondly, BTBench exhibits a wider performance distribution compared to SPEC CPU2017. These observations suggest that using BTBench for assessing the binary translator is a preferable option.

C. Translation Overhead

We employed the Linux Perf tool to sample the execution of binary translators. When the binary translator is translating input instructions, the program counter (PC) points to the code segment of the binary translator. Otherwise, the PC resides in code cache addresses. These two addresses can be distinguished based on the memory layout of the translator.

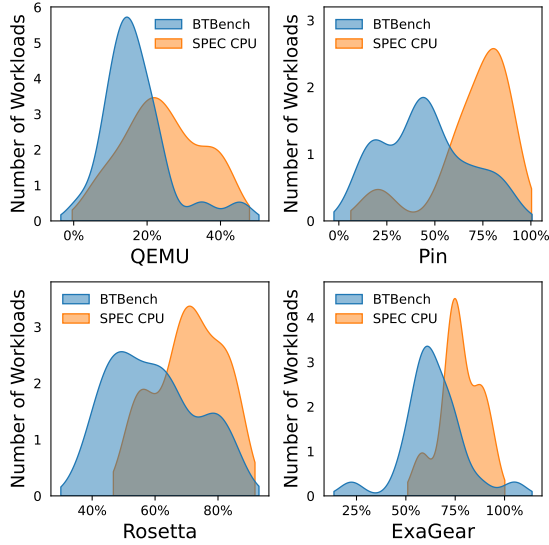


Fig. 3: Performance distribution of four binary translators on BTBench and SPEC CPU, peaks reflect the performance of most workloads, flatter and wider mean broader performance coverage.

The translation time is then determined by the ratio of the PC located in the translated component to the overall sampling of the program. The translation overhead can be calculated using Formulas 2 and 3. Using this method, translation overhead can also be obtained for closed-source translators.

$$RunTime_{translation} = \frac{Sample_{Translation}}{Sample_{All}} * RunTime_{BT} \quad (3)$$

The experimental results of the translation overhead, calculated using the aforementioned method, are presented in Figure 4a. According to the results, a translation overhead of less than 2.2% was observed during the execution of SPEC CPU2017. However, when running BTBench, the translation overhead is much higher, up to 83.0% (Pin). This demonstrates that the translation overhead plays a substantial role in the binary translator. In the case of Pin, three workloads exhibit translation overhead exceeding 100% (*gcc*: 311%, *pdfjs*: 244%, *script_compiler*: 709%), where the translation overhead dominates the overall performance. Additionally, the negligible translation overhead observed in SPEC CPU2017 may result in the oversight of optimizing the translation component. This discrepancy in translation overhead underscores the importance of enhancing the performance of the translation component for real-world applications.

We also investigated the factors contributing to the significant variations in translation overhead. The size of a program’s instruction footprint primarily affects the translation overhead since the translation algorithm of the binary translator is fixed. Consequently, evaluating the size of the workload’s instruction footprint provides insight into the extent of the translation overhead. Using the Pin instrumentation tool, we assessed the instruction footprint size of the workloads by counting the

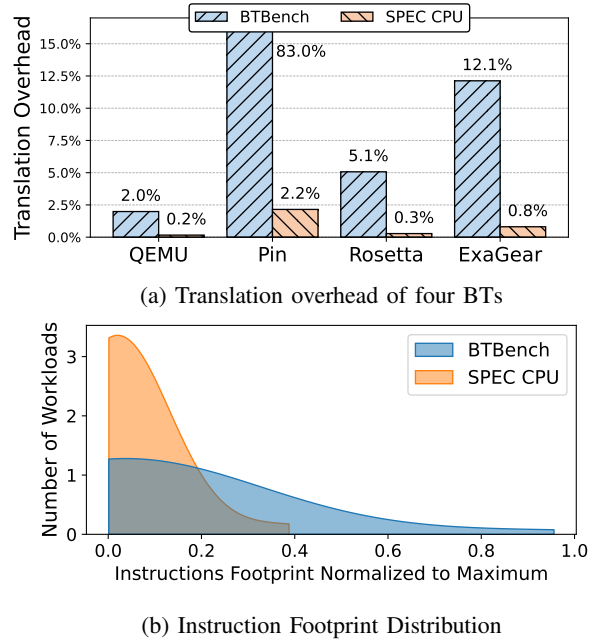


Fig. 4: Translation overhead and instruction footprint compare of SPEC CPU2017 and BTBench

number of unique instructions executed at different addresses and modified instructions executed at the same address. The results of the instruction footprint size are presented in Table I, and its distribution is illustrated in Figure 4b. These results indicate that the instruction footprint of BTBench greatly surpasses that of SPEC CPU2017, confirming the increased translation overhead observed from BTBench.

D. Code Lookup Overhead

Numerous studies have concentrated on optimizing code lookup algorithms in binary translators [20], [40]–[43]. In this paper, we examine the performance of code lookup in these four translators while conducting a thorough analysis utilizing both BTBench and SPEC CPU2017 benchmarks.

The assessment of the code lookup overhead faces two main challenges. **(a)** The code lookup in binary translation primarily originates from the input indirect jump instructions. Typically, the code lookup of a binary translator is integrated directly into the generated code of input indirect jump instructions [20]. Consequently, the execution of the code lookup and the execution of the translated code are closely intertwined. This tight coupling of instructions also presents challenges in determining whether the translator is performing code lookup, making it more difficult to estimate its impact on performance. **(b)** The code lookup in binary translation functions similarly to a hash lookup. One way to measure its performance is by calculating its hit rate, which indicates the ratio of successful searches. In terms of overall performance impact, it corresponds to the number of lookup misses within a given number of dynamic instructions. Although it is feasible to modify the source code of the open-source QEMU to assess

the number of lookup misses, it is not possible to conduct the same evaluation for the other three closed-source translators.

To tackle these challenges, we employed reverse engineering techniques to acquire the code lookup algorithms employed by the three proprietary translators. Firstly, the code lookup algorithm for binary translation can be discovered by constructing different types of indirect jump patterns, including call and jump instructions, as well as single and multiple jump targets. Subsequently, the execution flow of the translated instructions can be observed using gdb. Furthermore, we have developed a code lookup emulator using Pin tool. This emulator instruments indirect jump instructions in workloads and utilizes code lookup algorithms to calculate the miss ratio.

This approach enabled us to systematically assess the performance of the code lookup process and obtain valuable insights into the behavior of the four binary translators during the execution of BTBench and SPEC CPU2017.

In this study, we utilize Missed Predictions per Million Instructions (MPMI) as the performance metric. The results shown in Figure 5a revealed the following findings: **(a)** Rosetta 2 has the highest performance among the four translators, whereas QEMU and Pin occupied intermediate positions and ExaGear exhibited the lowest performance. **(b)** The BTBench suite places significantly higher demands on the code lookup process compared to SPEC CPU2017. It is observed that the MPMI is approximately double that observed in SPEC CPU.

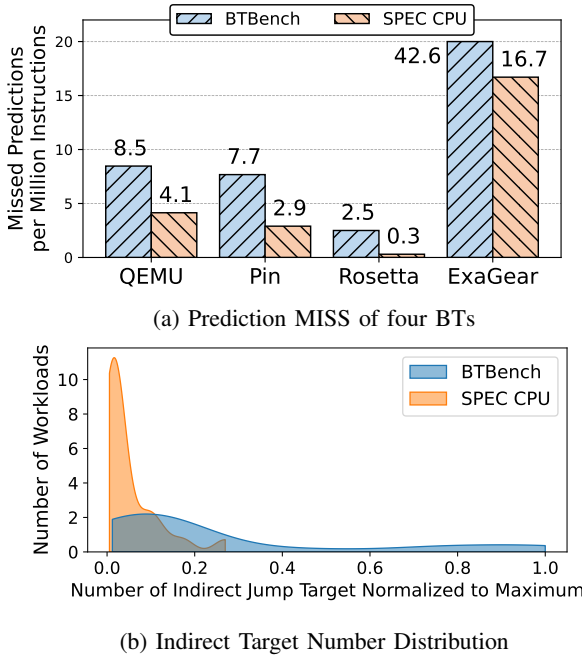


Fig. 5: Code lookup overhead compare of SPEC CPU2017 and BTBench

The performance of the code lookup algorithm is primarily influenced by the number of indirect jump targets. To investigate this aspect, we analyzed and graphed the distribution of indirect jump targets for both BTBench and SPEC CPU2017,

as depicted in Figure 5. The result revealed that BTBench presents a more extensive range of indirect jump targets compared to SPEC CPU2017, making it more suitable for assessing the performance of a binary translator.

It is worth mentioning that a well-designed code lookup algorithm can significantly enhance the performance of SPEC CPU [36]. However, the same algorithm may lead to significant performance degradation when running BTBench. This highlights the urgent need for optimizing code lookup algorithms specifically for different workloads.

To validate the accuracy of the algorithm inferred from reverse engineering, we performed supplementary experiments. We utilized QEMU, which operates on the AArch64 ISA, to execute ExaGear and capture the complete instruction stream during its execution. Through analyzing this stream, we could identify any search miss in the code lookup. By comparing the results with those obtained from the code lookup emulator, we were able to confirm the precision of the result. Unfortunately, our analysis was limited to ExaGear, as QEMU failed to execute Pin and Rosetta 2 due to functional incompleteness.

Figure 6 presents an example of the hash lookup instructions used by ExaGear for code lookup. By analyzing the tail branch directions of the instruction stream from ExaGear, we can determine the number of misses. The results obtained from this method are compared with the data obtained from the emulator, as shown in Table VI. The comparison reveals a strong similarity in the outcomes of the aforementioned two methods. Hence, the reliability of the results obtained using the code lookup emulation tool is confirmed.

```

add x1, sp, #0x1000
and x2, x18, #0x7fc
add x2, x1, x2, lsl #2
ldp x3, x4, [x2]
cmp x18, x4
b.ne miss_code
br x3

```

```

x1 = hash_table_addr
target_pc_hash = target_pc & 0x7fc
hash_item_addr = target_pc_hash + (hash_pc << 2)
x2 = host_pc (hash_item_addr[0])
x4 = guest_pc (hash_item_addr[1])
if guest_pc != pc, goto miss_code
else goto host_pc

```

Fig. 6: Sample Code Snippet for Code Lookup of ExaGear

TABLE VI: Missed Predictions per Million Instructions Evaluated by Two Methods

Benchmark	Instruction Trace Statistics	Simulation Tool
BTBench	38.5	42.6
SPEC CPU 2017	14.6	16.7

E. SMC Overhead

To investigate the impact of just-in-time (JIT) programs on the performance of binary translators, we categorized BTBench into two groups: *BTBench with JIT* and *BTBench without JIT*. The performance results are depicted in Figure 7. It is evident that Pin exhibits inferior performance on *BTBench with JIT*, and Exagear also displays a certain decrease in performance. The performance of handling SMC is affected by the retranslation overhead caused by SMC [72]. By comparing these findings with those in section IV-C, it is apparent that Pin

has the worst translation performance, followed by Exagear. This explains the inferior performance of Pin and Exagear on *BTBench with JIT*.

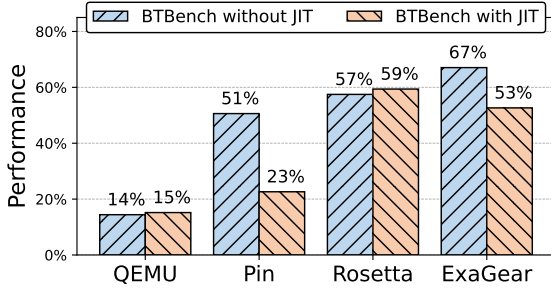


Fig. 7: Comparison of JIT Performance

Scripting languages commonly used in software rapid prototyping tools, such as JavaScript [2], [22], PHP [1], [31], [62], and PyPy [45], [46]. The execution of these languages is facilitated by just-in-time (JIT) compilers, which enhance the language performance.

The extensive usage and subpar performance of JIT highlight the need to include SMC programs in the evaluation of binary translator performance. Additionally, SPEC CPU does not include SMC, whereas BTBench incorporates seven SMC workloads, making it a more suitable option for assessing the performance of a binary translator.

F. Performance of Translated Code

The performance of the translated code can be assessed through the execution time of the translated code, which is challenging to precisely determine in binary translators. Workloads with minimal translation overhead and lacking SMC can more accurately demonstrate the performance of the translated code. The results of the above analysis indicate that the overhead of translation, code lookup, and SMC are minimal on SPEC CPU and part of the BTBench workloads. This finding suggests that the performance of SPEC CPU is mainly influenced by the performance of code produced by the translator.

Figure 2 demonstrates the superior performance of Pin compared to the other three translators on SPEC CPU, with Exe gear closely following behind. By examining the data from Figure 4a in conjunction with this observation, it can be inferred that Pin and Exe gear invest more time in the translation process, incurring higher translation overhead, to generate code of higher performance. In conclusion, SPEC CPU proves to be more proficient in assessing the performance of the generated code, while BTBench provides a more comprehensive assessment.

G. Code Coverage

We assessed the code coverage of the open-source QEMU through Gcov [27]. The results, shown in Table VII, indicate that BTBench achieved a higher code coverage compared to SPEC CPU2017, surpassing it in terms of both covered lines

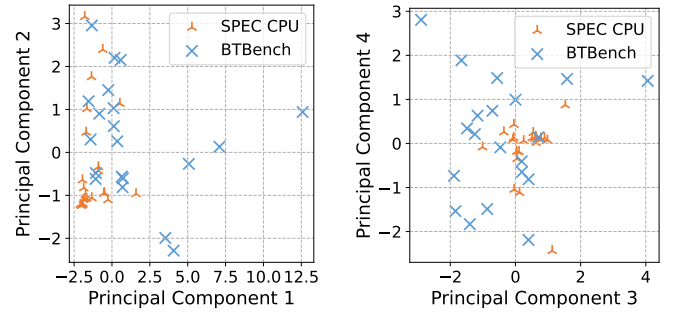


Fig. 8: PCA of SPEC CPU2017 and BTBench

and branches. Based on the result, it is suggested that BTBench outperforms SPEC CPU 2017 in terms of the coverage of the binary translation component.

TABLE VII: Code Coverage of QEMU Measured by Gcov

Benchmark	Lines Coverage	Branches Coverage
BTBench	36.0%	27.1%
SPEC CPU 2017	28.9%	21.0%

H. Principal Components Analysis

We used Principal Component Analysis (PCA) to reveal significant differences among the individual constituent benchmarks. PCA is a multivariate statistical method that aims to reduce a high-dimensional domain with correlated attributes to a lower-dimensional space. In this study, we utilized the indicators described in earlier sections to thoroughly analyze the benchmarks. The composite ensemble of metrics encompasses a total of 14 distinct features: static return/indirect call/indirect jump instruction number, static return/indirect call/indirect jump target address number, dynamic return/indirect call/indirect jump instruction count(normalized to the total number of instructions), instruction footprint, SMC number, translated instruction number, missed predictions per million instructions, translation overhead, performance.

The computation of PCA results in the identification of four principal components. The hierarchical ranking of these components is based on their relative significance. The scatter plot in Figure 8 visually represents the dispersion pattern of the benchmarks, showcasing their variance in a two-dimensional spectrum. The greater the spatial separation between two benchmarks, the more significant the metric disparity. Importantly, if two benchmarks assume distinct positions in either graph, they are considered divergent.

Observing Figure 8, it is apparent that: **(a)** The attributes of BTBench do not overlap with one another, indicating that each workload in BTBench exhibits distinct performance characteristics. In contrast, the characteristics of SPEC CPU2017 are more concentrated, implying a high degree of similarity. **(b)** The distribution of features in BTBench is more extensive in comparison to SPEC CPU2017. This suggests that BTBench

is more effective in encompassing the feature space related to binary translation performance. This disparity highlights the enhanced capacity of BTBench in assessing the performance of binary translators.

V. RELATED WORK

Vits [77], *SimBench* [70] and *Hyperbench* [74] are micro-benchmarks that focus on system-mode binary translation. These benchmarks measure the time taken for micro-operations such as TLB flush or external I/O. BTBench can also be used to evaluate the performance of system-mode BT, this work mainly focuses on user-mode BT.

PerfDBT [75] aims to speed up the performance testing of BT by profiling input programs like SPEC CPU and generating shorter runtime benchmarks. Nonetheless, *PerfDBT* only considers the code execution aspect and only represents the performance of applications similar to SPEC CPU.

Ruiz-Alvarez et al. [60] and Moore et al. [51] investigate the impact of binary translation on processor memory hierarchy, such as cache and translation lookaside buffer.

Edson et al. [10] and Surya Tej et al. [53] utilize *StarBT* and *DynamoRIO* as translators, respectively, and conduct a detailed performance analysis using SPEC CPU 2000/2006 benchmarks. Both translators use x86 as their host ISA and guest ISA, their performance results in the article indicate similarity to Pin. However, according to the findings of this study, Pin exhibited notable performance variations on the BTBench and SPEC CPU benchmarks. This suggests that depending exclusively on the SPEC CPU benchmark for performance analysis and optimization of binary translation may not be dependable.

VI. CONCLUSION

In this study, we conducted a comprehensive analysis of the limitations inherent in current binary translation benchmarks. Subsequently, we introduce BTBench, a benchmark specifically tailored for binary translation. To the best of our knowledge, BTBench is the first benchmark of its kind. The detailed analysis demonstrates that BTBench provides a more comprehensive evaluation of various aspects of binary translators in translation overhead, code lookup, SMC processing, and translated code quality, compared to SPEC CPU which only partially reflects the code quality generated by BT.

BTBench effectively reveals potential performance flaws that CPU benchmarks may overlook. For instance, due to insufficient performance in the translation portion, Pin shows a performance difference of 36.4% between BTBench and SPEC CPU. This fact goes completely unrecognized by SPEC CPU. Therefore, BTBench plays a crucial role in the comprehensive evaluation and optimization of binary translation performance.

ACKNOWLEDGEMENT

We are grateful for the invaluable comments and suggestions provided by the reviewers. This work is supported by the National Key Research and Development Program of China (under Grant 2022YFB3105103).

REFERENCES

- [1] K. Adams, J. Evans, B. Maher, G. Ottoni, A. Paroski, B. Simmers, E. Smith, and O. Yamauchi, "The hip-hop virtual machine," in *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*, 2014, pp. 777–790.
- [2] W. Ahn, J. Choi, T. Shull, M. J. Garzarán, and J. Torrellas, "Improving javascript performance by deconstructing the type system," *ACM SIGPLAN Notices*, vol. 49, no. 6, pp. 496–507, 2014.
- [3] E. R. Altman, D. Kaeli, and Y. Sheffer, "Welcome to the opportunities of binary translation," *Computer*, vol. 33, no. 3, pp. 40–45, Mar. 2000, number: 3.
- [4] Apple, "About the rosetta translation environment," 2021, accessed: 2023-3-30. [Online]. Available: <https://developer.apple.com/documentation/apple-silicon/about-the-rosetta-translation-environment>
- [5] Apple, "Running intel binaries in linux vms with rosetta," 2023, accessed: 2023-10-30. [Online]. Available: https://developer.apple.com/documentation/virtualization/running_intel_binaries_in_linux_vms_with_rosetta
- [6] J. Baker, "Buntdb is a low-level, in-memory, key/value store in pure go," 2023. [Online]. Available: <https://github.com/tidwall/buntdb>
- [7] V. Bala, E. Duesterwald, and S. Banerjia, "Dynamo: a transparent dynamic optimization system," in *SIGP*, 2000.
- [8] L. Baraz, T. Devor, O. Etzion, S. Goldenberg, A. Skaletsky, Y. Wang, and Y. Zemach, "Ia-32 execution layer: a two-phase dynamic translator designed to support ia-32 applications on itanium/spl reg/-based systems," in *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture*, 2003. *MICRO-36.*, 2003, pp. 191–201.
- [9] F. Bellard, "Qemu, a fast and portable dynamic translator," in *Proceedings of the FREENIX Track: 2005 USENIX Annual Technical Conference, April 10-15, 2005*. Anaheim, CA, USA: USENIX, 2005, pp. 41–46. [Online]. Available: <http://www.usenix.org/events/usenix05/tech/freenix/bellard.html>
- [10] E. Borin and Y. Wu, "Characterization of dbt overhead," in *2009 IEEE International Symposium on Workload Characterization (IISWC)*, 2009, pp. 178–187.
- [11] D. Bruening, T. Garnett, and S. Amarasinghe, "An infrastructure for adaptive dynamic optimization," in *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*, ser. CGO '03. USA: IEEE Computer Society, 2003, p. 265–275.
- [12] J. Bucek, K.-D. Lange, and J. v. Kistowski, "Spec cpu2017: Next-generation compute benchmark," in *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 41–42. [Online]. Available: <https://doi.org/10.1145/3185768.3185771>
- [13] T. E. Carlson, W. Heirman, S. Eyerman, I. Hur, and L. Eeckhout, "An evaluation of high-level mechanistic core models," *ACM Transactions on Architecture and Code Optimization (TACO)*, 2014.
- [14] J. Cesar, "Svgbob can create a nice graphical representation of your text diagrams," 2023. [Online]. Available: <https://github.com/ivanceras/svgbob>
- [15] L. Collin and J. T. Jia, "Xz utils," 2023. [Online]. Available: <https://tukaani.org/xz/>
- [16] E. G. Cota, P. Bonzini, A. Béné, and L. P. Carloni, "Cross-isa machine emulation for multicores," in *Proceedings of the 2017 International Symposium on Code Generation and Optimization*, ser. CGO '17. IEEE Press, 2017, p. 210–220.
- [17] E. G. Cota and L. P. Carloni, "Cross-isa machine instrumentation using fast and scalable dynamic binary translation," in *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 74–87. [Online]. Available: <https://doi.org/10.1145/3313808.3313811>
- [18] M. Cunha, N. Fournel, and F. Pétrot, "Collecting traces in dynamic binary translation based virtual prototyping platforms," in *Proceedings of the 2015 Workshop on Rapid Simulation and Performance Evaluation: Methods and Tools*, ser. RAPIDO '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: <https://doi.org/10.1145/2693433.2693437>
- [19] A. d'Antras, C. Gorgovan, J. Garside, and M. Luján, "Low overhead dynamic binary translation on arm," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2017, pp. 333–346.

- [20] A. d’Antras, C. Gorgovan, J. Garside, and M. Luján, “Optimizing Indirect Branches in Dynamic Binary Translators,” *ACM Trans. Archit. Code Optim.*, vol. 13, no. 1, Apr. 2016, place: New York, NY, USA Publisher: Association for Computing Machinery. [Online]. Available: <https://doi.org/10.1145/2866573>
- [21] J. C. Dehnert, B. K. Grant, J. P. Banning, R. Johnson, T. Kistler, A. Klaiber, and J. Mattson, “The transmeta code morphing™ software: Using speculation, recovery, and adaptive retranslation to address real-life challenges,” in *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*, ser. CGO ’03. USA: IEEE Computer Society, 2003, p. 15–24.
- [22] M. Dhok, M. K. Ramanathan, and N. Sinha, “Type-aware concolic testing of javascript programs,” in *Proceedings of the 38th International Conference on Software Engineering*, 2016, pp. 168–179.
- [23] ego, “Go efficient multilingual nlp and text segmentation,” 2023. [Online]. Available: <https://github.com/go-ego/gse>
- [24] A. Engelke, D. Okwieka, and M. Schulz, “Efficient LLVM-based dynamic binary translation,” in *Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. Virtual USA: ACM, Apr. 2021, pp. 165–171. [Online]. Available: <https://dl.acm.org/doi/10.1145/3453933.3454022>
- [25] EvanLi, “Github-ranking,” <https://github.com/EvanLi/Github-Ranking>, 2023.
- [26] J. Elyofson and P. Lam, “Detecting unread memory using dynamic binary translation,” in *Runtime Verification*, S. Qadeer and S. Tasiran, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 49–63.
- [27] Free Software Foundation, Inc., “gcov—a test coverage program,” 2023. [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc/Gcov.html>
- [28] S.-Y. Fu, D.-Y. Hong, J.-J. Wu, P. Liu, and W.-C. Hsu, “Simd code translation in an enhanced hqemu,” in *2015 IEEE 21st International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2015, pp. 507–514.
- [29] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. Pugsley, and J. Kim, “The Championship Simulator: Architectural Simulation for Education and Competition,” 2022.
- [30] Google, “Octane benchmark,” 2022. [Online]. Available: <https://developers.google.com/octane>
- [31] D. Gope, D. J. Schlais, and M. H. Lipasti, “Architectural support for server-side php processing,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2017, pp. 507–520.
- [32] C. Gorgovan, A. d’Antras, and M. Luján, “Mambo: A low-overhead dynamic binary modification tool for arm,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 13, no. 1, pp. 1–26, 2016.
- [33] R. Gouicem, D. Sprokholt, J. Ruehl, R. C. Rocha, T. Spink, S. Chakraborty, and P. Bhatotia, “Risotto: A dynamic binary translator for weak memory model architectures,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2022, pp. 107–122.
- [34] B. Hawkins, B. Demsky, D. Bruening, and Q. Zhao, “Optimizing binary translation of dynamically generated code,” in *2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2015, pp. 68–78.
- [35] J. L. Hennessy and D. A. Patterson, “A new golden age for computer architecture,” *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019.
- [36] J. D. Hiser, D. Williams, W. Hu, J. W. Davidson, J. Mars, and B. R. Childers, “Evaluating indirect branch handling mechanisms in software dynamic translation systems,” in *International Symposium on Code Generation and Optimization (CGO’07)*. IEEE, 2007, pp. 61–73.
- [37] D.-Y. Hong, C.-C. Hsu, P.-C. Yew, J.-J. Wu, W.-C. Hsu, P. Liu, C.-M. Wang, and Y.-C. Chung, “Hqemu: A multi-threaded and retargetable dynamic binary translator on multicores,” in *Proceedings of the Tenth International Symposium on Code Generation and Optimization*, ser. CGO ’12. New York, NY, USA: Association for Computing Machinery, 2012, p. 104–113. [Online]. Available: <https://doi.org/10.1145/2259016.2259030>
- [38] Huawei, “Huawei kunpeng exagear,” 2022, accessed: 2023-3-30. [Online]. Available: <https://mirrors.huaweicloud.com/kunpeng/archive/ExaGear/>
- [39] Huawei, “Dynamic binary translator (exagear),” <https://www.hikunpeng.com/developer/devkit/exagear>, 2023, [Accessed 27-10-2023].
- [40] N. Jia, C. Yang, Y. He, and X. Cheng, “Dtt: Program structure-aware indirect branch optimization via direct-tpc-table in dbt system,” in *Proceedings of the 11th ACM Conference on Computing Frontiers*, 2014, pp. 1–10.
- [41] N. Jia, C. Yang, Y. He, and X. Cheng, “Sptu: Improving dynamic binary translation through software prediction with target updating,” in *Proceedings of International Conference on Systems and Storage*, 2014, pp. 1–12.
- [42] N. Jia, C. Yang, J. Wang, D. Tong, and K. Wang, “Spire: improving dynamic binary translation through spe-indexed indirect branch redirecting,” *ACM SIGPLAN Notices*, vol. 48, no. 7, pp. 1–12, 2013.
- [43] H.-S. Kim and J. E. Smith, “Hardware support for control transfers in code caches,” in *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36*. IEEE, 2003, pp. 253–264.
- [44] J. Köster, “Rust-bio: a fast and safe bioinformatics library,” *Bioinformatics*, vol. 32, no. 3, pp. 444–446, 2016.
- [45] W. T. Lavrijsen and A. Dutta, “High-performance python-c++ bindings with pypy and cling,” in *2016 6th Workshop on Python for High-Performance and Scientific Computing (PyHPC)*. IEEE, 2016, pp. 27–35.
- [46] Y. Li and Z. M. Jiang, “Assessing and optimizing the performance impact of the just-in-time configuration parameters-a case study on pypy,” *Empirical Software Engineering*, vol. 24, pp. 2323–2363, 2019.
- [47] N. Lohmann, “Json for modern c++,” 2023. [Online]. Available: <https://github.com/nlohmann/json>
- [48] Loongson Technology, “Loongarch documentation,” <https://loongson.github.io/LoongArch-Documentation/>, 2022.
- [49] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, “Pin: building customized program analysis tools with dynamic instrumentation,” *Acm sigplan notices*, vol. 40, no. 6, pp. 190–200, 2005.
- [50] I. Molnar, “Performance counters for linux,” 2009, [Online]. Accessed on 2022-02-23. [Online]. Available: <https://lwn.net/Articles/337493/>
- [51] R. W. Moore, J. A. Baiocchi, B. R. Childers, J. W. Davidson, and J. D. Hiser, “Addressing the challenges of dbt for the arm architecture,” *Acm sigplan notices*, vol. 44, no. 7, pp. 147–156, 2009.
- [52] N. Nethercote and J. Seward, “Valgrind: A framework for heavyweight dynamic binary instrumentation,” *SIGPLAN Not.*, vol. 42, no. 6, p. 89–100, jun 2007. [Online]. Available: <https://doi.org/10.1145/1273442.1250746>
- [53] S. T. Nimmakayala, “Exploring causes of performance overhead during dynamic binary translation,” Ph.D. dissertation, University of Kansas, 2015.
- [54] G. Niu, F. Zhang, and X. Li, “Eliminate the Overhead of Interrupt Checking in Full-System Dynamic Binary Translator,” in *Proceedings of the 15th ACM International Conference on Systems and Storage*, ser. SYSTOR ’22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 1–12, event-place: Haifa, Israel. [Online]. Available: <https://doi.org/10.1145/3534056.3534939>
- [55] H. Patil, A. Isaev, W. Heirman, A. Sabu, A. Hajiabadi, and T. E. Carlson, “Elfies: Executable region checkpoints for performance analysis and simulation,” in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2021, pp. 126–136.
- [56] H. Patil, C. Pereira, M. Stallcup, G. Lueck, and J. Cownie, “Pinplay: a framework for deterministic replay and reproducible analysis of parallel programs,” in *Proceedings of the 8th annual IEEE/ACM international symposium on Code generation and optimization*, 2010, pp. 2–11.
- [57] PHP, “The php interpreter,” 2023. [Online]. Available: <https://github.com/php/php-src>
- [58] phpbench, “Php benchmarking framework,” 2023. [Online]. Available: <https://github.com/phpbench/phpbench>
- [59] R. C. Rocha, D. Sprokholt, M. Fink, R. Gouicem, T. Spink, S. Chakraborty, and P. Bhatotia, “Lasagne: a static binary translator for weak memory model architectures,” in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022, pp. 888–902.
- [60] A. Ruiz-Alvarez and K. Hazelwood, “Evaluating the impact of dynamic binary translation systems on hardware cache performance,” in *2008 IEEE International Symposium on Workload Characterization*. IEEE, 2008, pp. 131–140.
- [61] rust lang, “GitHub - rust-lang/mdBook: Create book from markdown files. Like Gitbook but implemented in Rust — github.com,” <https://github.com/rust-lang/mdBook>, [Accessed 16-07-2023].

- [62] H. Samimi, M. Schäfer, S. Artzi, T. Millstein, F. Tip, and L. Hendren, “Automated repair of html generation errors in php applications using string constraint solving,” in *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 277–287.
- [63] B.-Y. Shen, W.-C. Hsu, and W. Yang, “A retargetable static binary translator for the arm architecture,” *ACM Trans. Archit. Code Optim.*, vol. 11, no. 2, jun 2014. [Online]. Available: <https://doi.org/10.1145/2629335>
- [64] SPEC, “502.gcc, spec cpu@2017 benchmark description,” 2023. [Online]. Available: https://www.spec.org/cpu2017/Docs/benchmarks/502.gcc_r.html
- [65] SPEC, “502.perlbench, spec cpu@2017 benchmark description,” 2023. [Online]. Available: https://www.spec.org/cpu2017/Docs/benchmarks/500.perlbench_r.html
- [66] SPEC, “519.lbm, spec cpu@2017 benchmark description,” 2023. [Online]. Available: https://www.spec.org/cpu2017/Docs/benchmarks/519.lbm_r.html
- [67] Standard Performance Evaluation Corporation, “Spec cpu benchmark suites,” 2023. [Online]. Available: <https://www.spec.org/cpu/>
- [68] G. team, “GCC, the GNU Compiler Collection - GNU Project — gcc.gnu.org,” <https://gcc.gnu.org/>, [Accessed 16-07-2023].
- [69] tiobe, “TIOBE Index - TIOBE — tiobe.com,” <https://www.tiobe.com/tiobe-index/>, [Accessed 16-07-2023].
- [70] H. Wagstaff, B. Bodin, T. Spink, and B. Franke, “Simbench: A portable benchmarking methodology for full-system simulators,” in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2017, pp. 217–226.
- [71] W. Wang, “Helper function inlining in dynamic binary translation,” in *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*, 2021, pp. 107–118.
- [72] W. Wang, J. Wu, X. Gong, T. Li, and P.-C. Yew, “Improving Dynamically-Generated Code Performance on Dynamic Binary Translators,” in *Proceedings of the 14th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE ’18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 17–30. [Online]. Available: <https://dl.acm.org/doi/10.1145/3186411.3186413>
- [73] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, “The risc-v instruction set manual, volume i: Base user-level isa,” *EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2011-62*, vol. 116, pp. 1–32, 2011.
- [74] S. Wei, K. Zhang, and B. Tu, “Hyperbench: A benchmark suite for virtualization capabilities,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 3, no. 2, pp. 1–22, 2019.
- [75] J. Wu, J. Dong, R. Fang, W. Wang, and D. Zuo, “Perfdbt: Efficient performance regression testing of dynamic binary translation,” in *2020 IEEE 38th International Conference on Computer Design (ICCD)*. IEEE, 2020, pp. 389–392.
- [76] J. Yiu, “Armv8-m architecture technical overview,” *ARM white paper*, 2015.
- [77] P. Yuan, C. Ding, L. Cheng, S. Li, H. Jin, and W. Cao, “Vits test suit: a micro-benchmark for evaluating performance isolation of virtualization systems,” in *2010 IEEE 7th International Conference on E-Business Engineering*. IEEE, 2010, pp. 132–139.
- [78] J. Zeng, Y. Fu, and Z. Lin, “Pemu: A pin highly compatible out-of-vm dynamic binary instrumentation framework,” in *Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, 2015, pp. 147–160.